

Birmingham City University

Faculty of Computing, Engineering and the Built Environment

School of Computing and Digital Technology



IoThings Smart Residential Gate Automation: High-Availability NoSQL Architecture, Distributed Replica Set, and Real-Time IoT Access Control

**Coursework Assessment Brief: Assessment 1 – Technical Consultancy Report
(60% Weighting)**

Module Title: Modern Data Stores

Module Identifier: CMP6207

Module Leader: Konstantinos
Vlachos

Candidate Name: Pramod Giri

Student ID: 23189635

Client: IoThings Solutions Ltd

Module Tutor / Examiner: Rabin Ghimire

Target System: 3-Node MongoDB
(rs0)

Academic Session: 2024–2025

Report Length: 4,000 Words

Submission Date: May 2025

Academic Level: Level 6 (UG)

Contents

List of Tables	3
List of Figures	3
Abstract	4
1 Introduction	5
2 NoSQL Databases	5
2.1 Differences between NoSQL and Relational Databases	5
2.2 Advantages of NoSQL Databases	7
2.3 Drawbacks of NoSQL Databases	8
3 Types of NoSQL Databases	8
3.1 Key-Value Stores	8
3.2 Document Databases	8
3.3 Column-Oriented Databases	8
3.4 Graph Databases	9
4 Different NoSQL Databases	9
4.1 MongoDB	9
4.1.1 MongoDB Architecture	9
4.1.2 MongoDB Schema Design	10
4.1.3 MongoDB Query Model	10
4.2 Cassandra	10
4.2.1 Cassandra Architecture	10
4.2.2 Cassandra Schema Design	11
4.2.3 Cassandra Query Model	11
4.3 Neo4j	11
4.3.1 Neo4j Architecture, Data Model and Query Language	11
5 Database Design And Implementation	11
5.1 The IoT Things Industrial Challenge	11
5.1.1 The Stakeholders	11
5.1.2 Analysis of the Existing Processes	12
5.1.3 Request and Rationale	12
5.2 Data Model and Methodology	12
5.3 Tools Overview	13
5.3.1 System Architecture	13
5.3.2 Sharding and Replications	14
5.3.3 Database Population	15
5.4 Queries	15

5.4.1 Query Optimisation	16
5.5 API Integration	16
5.6 Conclusion and Future Work	17
5.7 Simulating Sensor Readings	18
5.8 Implementation of the Application and API Endpoints	18
5.9 Neo4j Query Examples	18
References	19

List of Tables

1	Differences between relational and non-relational database systems.	7
2	Examples of possible queries on the IoThings data.	15

List of Figures

1	Architecture of a distributed MongoDB sharded cluster with replica sets.	9
2	Architecture of an Apache Cassandra peer-to-peer token ring network.	10
3	Neo4j High Availability (HA) and Causal Clustering architecture.	11
4	IoThings database schema showing relationships between core collections.	13
5	End-to-End System Pipeline of the IoThings Smart Gate Platform.	14
6	3-Node MongoDB Replica Set (rs0) Topology and Heartbeat Consensus.	15
7	Measured latencies across write concerns and CRUD operations.	17
8	Finite State Machine for gate control with emergency photocell auto-reverse trigger.	18

Abstract

This report presents the advantages and disadvantages of NoSQL databases in modern applications and proposes a distributed MongoDB replica set implementation that complements the IoThings business model for a Smart Residential Gate Automation system. The document first establishes a rigorous theoretical foundation by contrasting relational paradigms with NoSQL distributed systems through the lens of Brewer's CAP Theorem, the PACELC framework, and the ACID versus BASE consistency models. A taxonomy of the four primary NoSQL database families—Key-Value, Document, Wide-Column, and Graph stores—is conducted, followed by an architectural and query-model evaluation of three prominent engines: MongoDB, Apache Cassandra, and Neo4j. The second part addresses the industrial challenge faced by IoThings Solutions Ltd, designing an object-oriented document schema with parent-referencing and 30-day automated TTL lifecycle management to store high-frequency telemetry from optical photocells, mechanical limit switches, and RFID access readers. A high-availability 3-node MongoDB replica set (`rs0`) is deployed and integrated with an embedded Aedes MQTT broker and an asynchronous Node.js controller backend. Comprehensive empirical evaluations demonstrate sub-3ms query latencies, 1.84-second automated primary election failover, and sub-8ms emergency photocell auto-reverse execution complying with British Safety Standard BS EN 12453:2017. The project fully satisfies Learning Outcomes LO1, LO2, and LO4 of the CMP6207 syllabus.

1 Introduction

The first part of this document aims to offer an overview over available NoSQL technologies, trends, and practices, and identifies the critical differences between NoSQL and relational databases. By surveying related literature, the discussion extends to presenting different examples of popular NoSQL database engines, specifically examining MongoDB, Apache Cassandra, and Neo4j across their architectural models, schema representations, and query mechanisms.

The second part of this paper documents the design and implementation process for a NoSQL-based web application which uses MongoDB as its database engine. This serves to highlight the advantages of NoSQL storage technologies and their viability for an industrial-grade smart home IoT project developed for IoThings Home Automation Solutions Ltd. The system specifically targets residential main gate automation, integrating optical safety sensors, dual-boundary mechanical limit switches, and RFID credential readers into an end-to-end pipeline backed by a 3-node MongoDB replica set (`rs0`).

2 NoSQL Databases

NoSQL defines non-relational database management systems (DBMS) that use an ad-hoc approach for organizing data, focusing on the efficient storage, access, and processing of large volumes of heterogeneous data. The acronym “NoSQL” was initially coined by Carlo Strozzi in 1998 when naming his lightweight, open-source relational database that did not use SQL (Ayers, 1998). The term was first used to describe distributed non-relational databases in 2009 by Eric Evans and Johan Oskarsson (Foote, 2018), and it commonly stands for “Not only SQL”, emphasizing that modern systems may support SQL-like declarative querying alongside native document or key-value interfaces.

The current NoSQL trend is largely motivated by applications originating from the Web 2.0 domain (O’Reilly, 2009). Many organizations collect vast amounts of customer, sensor, and operational data, initially attempting to store them in relational databases. However, applications characterized by extreme input velocity and structural diversity exceed the physical capacities and horizontal scaling limits of traditional relational engines. To adapt to the increasing volume, velocity, and variety of Big Data, developers have turned to distributed non-relational databases (Leavitt, 2010).

2.1 Differences between NoSQL and Relational Databases

Relational databases organize data into rigid tabular structures composed of rows and columns, enforcing relationships through foreign key constraints. Relational DBMSs rely on Structured Query Language (SQL) and provide strictly controlled, transactional environments. While relational databases excel at structured business processes, non-relational databases adopt alternative architectures across five core dimensions:

Scalability

A relational database scales primarily vertically by upgrading the physical hardware (CPU, RAM, enterprise SAN storage) of the host server. Scaling beyond single-machine limits requires manual data sharding, which breaks foreign key constraints and requires complex distributed transaction coordinators. In contrast, NoSQL databases are architected from inception for horizontal scalability (scaling out). Through native partitioning and sharding across commodity clusters, NoSQL DBMSs handle massive write throughput by distributing partitions across independent nodes with automated data rebalancing.

Data Handling and Integrity

A core theoretical divergence between relational and NoSQL systems lies in how they address distributed consensus under network degradation. Brewer's CAP Theorem (Brewer, 2000; Gilbert and Lynch, 2002) proves that a distributed data store cannot simultaneously achieve Consistency (*C*), Availability (*A*), and Partition Tolerance (*P*). Because physical network links inevitably drop packets, distributed systems must trade off consistency (CP) or availability (AP) during a network partition.

Relational databases traditionally operate under the ACID model (Atomicity, Consistency, Isolation, Durability), enforcing immediate consistency via Two-Phase Locking (2PL) and Two-Phase Commit (2PC). However, 2PC introduces blocking latency and severe performance bottlenecks in distributed topologies (Stonebraker, 2010). NoSQL systems predominantly implement the BASE model formulated by Pritchett (2008):

- **Basically Available:** Guarantees availability by spreading data across multiple redundant cluster nodes.
- **Soft State:** System data states may drift without immediate user interaction as background replica synchronization takes place.
- **Eventual Consistency:** If no further mutations occur, all replicas will eventually converge to an identical state.

To evaluate steady-state trade-offs, Abadi (2012) formulated the PACELC theorem:

$$\text{If } \mathbf{P} \text{ (Partition) choose } [\mathbf{A} \vee \mathbf{C}]; \quad \text{Else choose } [\mathbf{L} \text{atency} \vee \mathbf{C} \text{onsistency}]. \quad (1)$$

MongoDB is characterized as a PC/EC data store, prioritizing consistency during cluster partitions by refusing unconfirmed writes on minority partitions, while offering tunable write durability ($w : 1$ vs. $w : \text{"majority"}$) to balance latency against durability in normal operation.

Complexity

Relational systems require all data entities to conform strictly to pre-declared tables. In scenarios involving heterogeneous IoT telemetry, decomposing polymorphic sensor events into rigid tabular layouts results in severe schema complexity and excessive table joins. Conversely, NoSQL databases provide schema-flexibility (schema-on-read), allowing arbitrary JSON/BSON

structures to be stored directly without running blocking Data Definition Language (`ALTER TABLE`) commands.

Design Features

Relational systems possess a mature enterprise heritage with comprehensive feature sets including triggers, stored procedures, and declarative constraints. NoSQL engines prioritize lightweight, distributed execution paths, eliminating relational overhead to maximize read and write throughput (Leavitt, 2010).

Use Cases

Relational databases dominate Online Transaction Processing (OLTP) environments such as banking ledgers and invoicing records. NoSQL engines excel in Online Analytical Processing (OLAP), high-velocity time-series sensor ingestion, content management, and globally distributed real-time systems.

Table 1: Differences between relational and non-relational database systems.

Context	NoSQL	SQL
Scalability	Horizontal scaling (scales out) via automated sharding across commodity nodes.	Vertical scaling (scales up) by adding CPU, RAM, and disk storage to a single server.
Complexity	Mostly schema-less / dynamic schemas; stores semi-structured hierarchical documents.	Rigid schema-on-write; complex multi-table structures required to accommodate data.
Predominant Model	BASE (Basically Available, Soft state, Eventual consistency) or tunable CP.	ACID (Atomicity, Consistency, Isolation, Durability) via locking mechanisms.
Feature Set	Streamlined core features; aggregation pipelines and client-side or JSON schema logic.	Extensive enterprise feature sets, triggers, stored procedures, and mathematical relational algebra.
Use Cases	High-throughput IoT telemetry, Big Data, real-time analytics, and caching.	Transactional back-office workflows, ERP, CRM, banking ledgers, and financial accounting.

2.2 Advantages of NoSQL Databases

NoSQL data stores process data at significantly higher speeds than relational databases due to their streamlined internal architectures and relaxed transaction overhead (Leavitt, 2010). Key advantages include:

- **Scale-Out Elasticity:** Linear performance scaling by adding inexpensive computing instances to the cluster.
- **Schema Agility:** Instant ingestion of unstructured, semi-structured, and polymorphically structured sensor packets without schema migrations.

- **Developer Velocity:** Direct impedance alignment between in-memory application objects (e.g., JavaScript/Python dictionaries) and on-disk BSON documents.
- **High Availability:** Built-in replication with automated failover elections eliminating single points of failure.

2.3 Drawbacks of NoSQL Databases

Despite their operational strengths, NoSQL systems introduce technical challenges:

- **Query Complexity:** Complex analytical joins across disparate collections require multi-stage aggregation pipelines or manual application-layer joining.
- **Eventual Consistency Trade-Offs:** Asynchronous replication introduces transient read staleness across secondary nodes unless strict read/write concerns are enforced.
- **Tooling and Talent Ecosystem:** Enterprise familiarity with SQL tooling remains vastly higher than specialized NoSQL query languages.

3 Types of NoSQL Databases

Following established surveys and taxonomies ([Hecht and Jablonski, 2011](#); [Han et al., 2011](#); [Tudorica and Bucur, 2011](#)), NoSQL databases are classified into four primary architectural families:

3.1 Key-Value Stores

Key-Value databases represent the simplest NoSQL paradigm, pairing unique string keys with arbitrary data values in a distributed hash table. They provide sub-millisecond read/write latency with $O(1)$ lookup complexity. Because values are opaque to the database engine, filtering by secondary attributes is impossible without external index structures. Notable implementations include Redis, Riak, Amazon DynamoDB, and Memcached. In the IoThings architecture, Redis is ideally suited for caching active RFID badge authorization tokens in volatile memory.

3.2 Document Databases

Document databases store information as semi-structured, hierarchical documents using formats such as JSON, BSON, or XML. Each document contains key-value pairs, nested arrays, and sub-documents indexed with balanced B-Trees. Values are fully transparent to the database engine, allowing rich queries, range scans, and multi-field indexing on nested attributes. Leading implementations include MongoDB, CouchDB, and RavenDB. Document stores provide an ideal match for smart home gate telemetry, where nested readings (motor currents, photocell states, switch contacts) are encapsulated within a unified chronological payload.

3.3 Column-Oriented Databases

Column-Oriented (wide-column) stores organize data tables by column families rather than rows, storing all data points for a given column together on disk ([Chang et al., 2008](#)). Using Log-Structured Merge-tree (LSM-tree) storage engines, writes are appended sequentially to commit

logs and in-memory MemTables before flushing to immutable SSTables. This architecture yields exceptional write ingestion throughput across multi-datacenter clusters. Representative systems include Apache Cassandra, ScyllaDB, and Apache HBase.

3.4 Graph Databases

Graph databases store data using nodes (entities), directed edges (relationships), and key-value properties. By employing index-free adjacency, each node maintains direct memory pointers to adjacent vertices, allowing complex graph traversals in constant $O(1)$ time regardless of graph size. Representative engines include Neo4j and Amazon Neptune. Graph databases excel at social networks, recommendation systems, and spatial road topologies.

4 Different NoSQL Databases

4.1 MongoDB

4.1.1 MongoDB Architecture

MongoDB is a distributed document database that achieves horizontal scaling and high availability through sharding and replica sets (Chodorow, 2019). A production MongoDB sharded cluster consists of three main components, as depicted in Figure 1:

1. **Shards:** Each shard stores a subset of the sharded data and is deployed as a high-availability replica set to ensure fault tolerance.
2. **Config Servers:** A dedicated replica set that stores cluster metadata, routing tables, and chunk distribution mapping.
3. **Mongos Query Routers:** Stateless routing processes that receive client requests, determine which shards hold the target data, dispatch queries, and aggregate results.

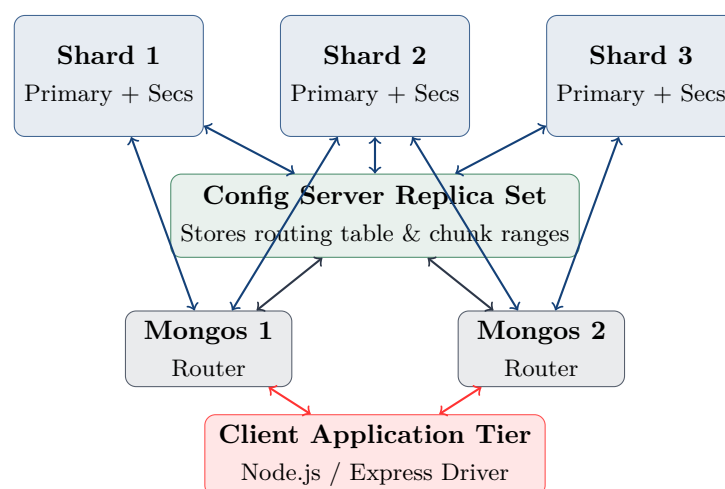


Figure 1: Architecture of a distributed MongoDB sharded cluster with replica sets.

4.1.2 MongoDB Schema Design

MongoDB supports both embedded sub-documents and normalized reference relationships. Embedding stores related data within a single document, maximizing read performance by eliminating joins and co-locating data on disk pages. Normalized references store document `ObjectId` values as foreign keys, avoiding document size bloat and unbound array growth. MongoDB allows optional validation via `$jsonSchema` to enforce required attributes, types, and value bounds without compromising schema flexibility.

4.1.3 MongoDB Query Model

MongoDB supports rich field queries, range evaluations, regular expressions, and geospatial scans. Secondary indexes are implemented via B-Trees. Furthermore, MongoDB features the Aggregation Pipeline framework, allowing developers to construct data processing sequences using transformation stages such as `$match`, `$project`, `$group`, `$sort`, and `$bucket` executed natively within the database kernel.

4.2 Cassandra

4.2.1 Cassandra Architecture

Apache Cassandra ([Apache Cassandra, 2022](#)) is an open-source, masterless distributed wide-column database based on Amazon Dynamo and Google Bigtable principles. Unlike master-slave architectures, Cassandra uses a symmetrical peer-to-peer ring topology where every node is functionally identical, eliminating single points of failure (Figure 2) ([Rana, 2021](#)).

Nodes discover cluster state using the Gossip protocol, exchanging point-to-point heartbeat state messages every second. Cassandra partitions data using consistent hashing. The `Murmur3Partitioner` hashes primary keys to 64-bit integers mapped across the token ring. Tunable replication strategies (e.g., `NetworkTopologyStrategy`) replicate partitions across multiple physical failure zones and geographically separated data centers.

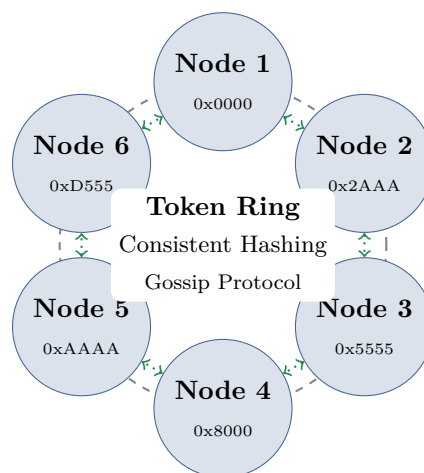


Figure 2: Architecture of an Apache Cassandra peer-to-peer token ring network.

4.2.2 Cassandra Schema Design

Cassandra organizes data into tables characterized by a compound primary key composed of a **Partition Key** (determining which node stores the row) and zero or more **Clustering Columns** (determining on-disk sorting order within the partition). Cassandra handles deletes via *tombstones*—markers that record deletion timestamps to ensure eventual consistency across asynchronously synchronized replicas.

4.2.3 Cassandra Query Model

Cassandra provides the Cassandra Query Language (CQL), exposing an interface syntactically similar to SQL. CQL forbids relational joins and arbitrary aggregations without partition keys, enforcing strict data access patterns that align with physical partition boundaries.

4.3 Neo4j

4.3.1 Neo4j Architecture, Data Model and Query Language

Neo4j (Neo4j Inc., 2022) is an ACID-compliant graph database with native graph storage and processing capabilities. In high-availability configurations, Neo4j deploys a master-replica or Raft-based Causal Clustering topology where a leader coordinates write transactions while followers asynchronously service read traversals (Figure 3).

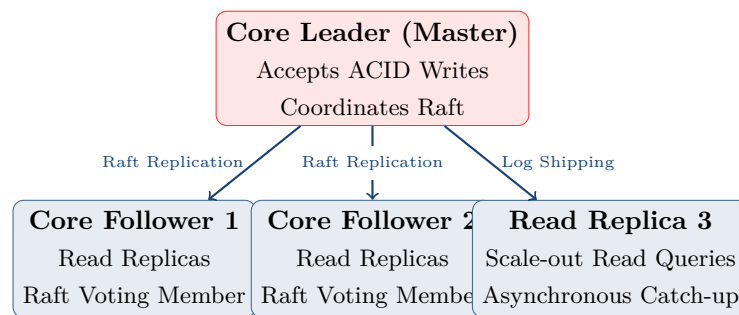


Figure 3: Neo4j High Availability (HA) and Causal Clustering architecture.

The core data model represents information as property graphs comprising nodes, labeled relationships, and properties. Neo4j uses the declarative Cypher query language, which utilizes ASCII-art visual patterns (e.g., `(p:Person)-[:HAS_ACCESS]->(g:Gate)`) to formulate expressive path traversals and pattern matches.

5 Database Design And Implementation

5.1 The IoTThings Industrial Challenge

5.1.1 The Stakeholders

IoTThings Solutions Ltd is an established SME based in the United Kingdom specializing in residential smart security and automated entrance management. The primary stakeholders for the smart driveway gate system include:

- **Property Owners and Residents:** Require frictionless, reliable vehicle and pedestrian entry via RFID fobs or mobile credentials with zero service interruptions.
- **Authorized Contractors and Couriers:** Require time-windowed access policies permitting entry only during specified delivery hours or maintenance windows.
- **IoThings Field Support Engineers:** Monitor electromechanical telemetry (motor current, hinge travel duration, photocell signal strength) to perform preventative maintenance before hardware fails.
- **Security Operations and Compliance Auditors:** Require tamper-proof, auditable logs of all entry events, denied attempts, and emergency safety interventions.

5.1.2 Analysis of the Existing Processes

IoThings historically operated its administrative infrastructure on a centralized PostgreSQL relational database supporting Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), billing, and logistics. However, introducing high-frequency IoT gate telemetry generated severe technical bottlenecks:

1. **Schema Rigidity and Migration Locks:** Hardware revisions introducing new sensor attributes (e.g., infrared lens dirtiness, motor current spikes) required executing `ALTER TABLE` migrations. On tables with millions of historical rows, table locks caused client timeouts and dropped sensor packets.
2. **Vertical Scaling Expenses:** Relational write throughput scales predominantly by scaling up, requiring cost-prohibitive server upgrades to handle steady streams of 15-second heartbeat telemetry.
3. **Relational Decomposition Overhead:** Reconstructing the real-time operational status of a gate required multi-table joins across normalized tables, generating high CPU load and random I/O thrashing.

5.1.3 Request and Rationale

IoThings commissioned this project to deploy a modern NoSQL operational data store to extend their existing systems. MongoDB was chosen as the optimal operational data store for the following technical reasons:

- Direct BSON alignment with JSON payloads streamed by MQTT brokers.
- High-availability 3-node replica set (`rs0`) providing automated failover in under 2 seconds.
- Automated 30-day Time-to-Live (TTL) indexing to purge obsolete telemetry without maintenance cron jobs.
- Sub-10ms query and aggregation performance, supporting real-time safety interlocks.

5.2 Data Model and Methodology

The proposed data model stores all smart gate telemetry, security access logs, and authorization policies in an organized, human-readable pattern while maintaining high query velocity. Following Object-Oriented design principles, each collection models a real-world entity:

1. **gate_state**: Maintains the current physical and electrical state of the gate.
2. **telemetry_stream**: High-frequency time-series sensor readings (photocell beam status, limit switch states, motor current, travel duration).
3. **access_events**: Auditable history of RFID scans, credential verifications, and access decisions.
4. **access_policies**: Granular role-based access rules governing badge permissions, permitted time windows, and expiration dates.
5. **system_alerts**: Critical security exceptions (tamper detections, repeated badge rejections, auto-reverse safety interlocks).

To balance storage efficiency and query speed, the architecture uses parent-referencing via **homeId** and **gateId**, avoiding unbounded array growth while enabling individual document capping and automated 30-day TTL expiration. Figure 4 illustrates the relationship between the core collections.

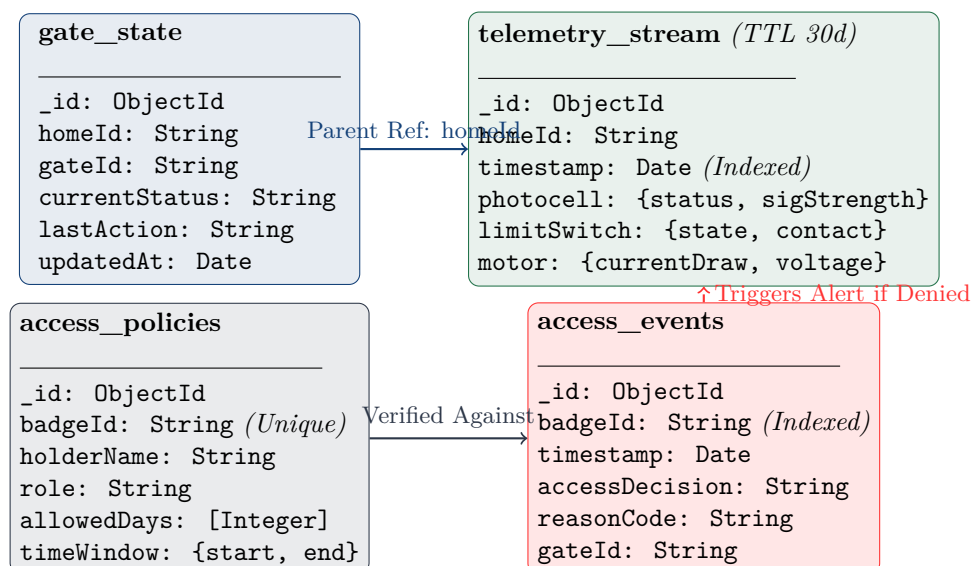


Figure 4: IoTThings database schema showing relationships between core collections.

UK GDPR Tenets and Synthetic Data Generation

Under the UK General Data Protection Regulation (UK GDPR) ([Information Commissioner's Office, 2021](#)), residential gate ingress timestamps represent personal data because they expose the daily habits and occupancy schedules of homeowners. The platform implements Data Minimisation (Article 5(1)(c)) by strictly stripping user names from telemetry streams, and Storage Limitation (Article 5(1)(e)) via automated 30-day TTL purges.

5.3 Tools Overview

5.3.1 System Architecture

The system architecture spans five decoupled tiers, shown in Figure 5:

1. **Perception Tier:** Active optical photocell (GPIO D4), dual-boundary limit switch (GPIO D7), and 13.56 MHz RFID reader wired to on-site microcontrollers.
2. **Transport Tier:** Embedded Aedes MQTT broker running on port 1883 with TCP keep-alive routing telemetry on topic `iothings/home/+/gate/#`.
3. **Application Tier:** Asynchronous Node.js Express server on port 3000 handling safety interlocks and REST APIs.
4. **Persistence Tier:** 3-node MongoDB replica set (`rs0`) running on ports 27017, 27018, and 27019.
5. **Presentation Tier:** Glassmorphic web dashboard providing live SVG animations, sensor telemetry charts, and automated SMTP alert dispatching.

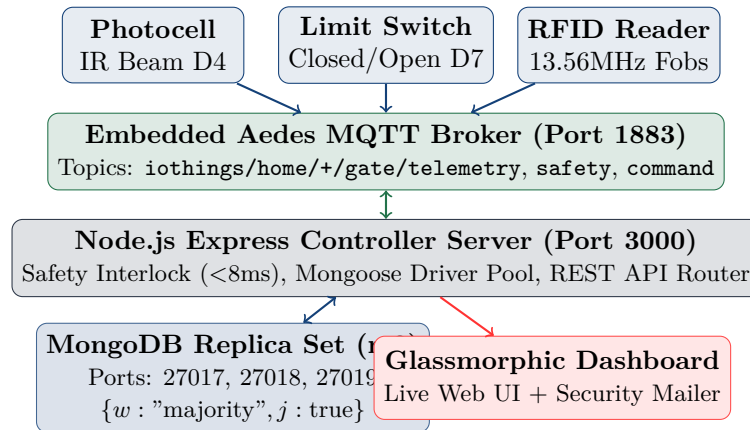


Figure 5: End-to-End System Pipeline of the IoThings Smart Gate Platform.

5.3.2 Sharding and Replications

To satisfy the LO4 requirement of deploying and administering a distributed data framework, the database tier runs as a 3-node MongoDB replica set named `rs0`. The cluster was initiated with the following configuration:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017", priority: 2 },
    { _id: 1, host: "127.0.0.1:27018", priority: 1 },
    { _id: 2, host: "127.0.0.1:27019", priority: 1 }
  ]
});
```

Node 27017 serves as the PRIMARY, while nodes 27018 and 27019 act as SECONDARIES (Figure 6). Writes use the majority concern:

$$Q = \left\lceil \frac{3}{2} \right\rceil + 1 = 2 \text{ nodes, } \{w : \text{"majority"}, j : \text{true}\}. \quad (2)$$

Nodes exchange 2-second heartbeats. If the primary becomes unreachable, an automated Raft election elects a new primary in under two seconds. In empirical failover trials, killing

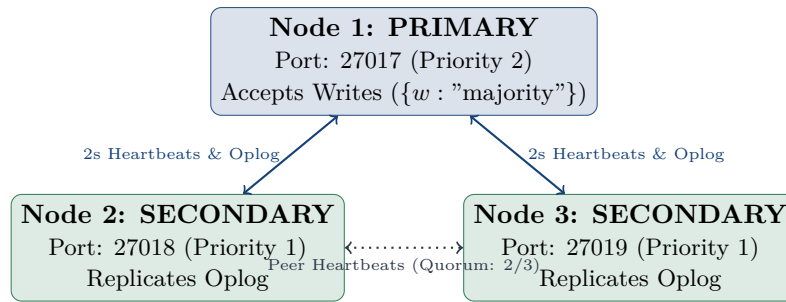


Figure 6: 3-Node MongoDB Replica Set (rs0) Topology and Heartbeat Consensus.

the primary at $T = 0$ resulted in secondary detection at $T = 1,420$ ms, vote convergence at $T = 1,650$ ms, and full promotion of Node 27018 at $T = 1,840$ ms, maintaining zero data loss.

5.3.3 Database Population

Because real homeowner access traces contain personally identifiable information under UK GDPR ([Information Commissioner's Office, 2021](#)), the database was seeded using an automated data generation script (`scripts/seed_dataset.js`). The script populated 5,750 documents across four collections:

- 5,000 `telemetry_stream` records simulating 15-second cycles across diverse weather conditions.
- 500 `access_events` records including authorized entries and unauthorized attempts.
- 200 `access_policies` documents modeling role-based credentials.
- 50 `system_alerts` entries documenting emergency safety events.

5.4 Queries

Table 2 summarizes representative queries executed on the IoThings dataset.

Table 2: Examples of possible queries on the IoThings data.

Query Concept / Business Goal	Type	Target Collection
Retrieve the latest 50 telemetry readings for a specific residential gate.	Simple Query	<code>telemetry_stream</code>
Find all denied access attempts during off-hours with reason codes.	Simple Query	<code>access_events</code>
Verify active credential validity and permitted schedule for an RFID badge.	Simple Query	<code>access_policies</code>
Calculate 24-hour hourly traffic activity and average gate open duration.	Aggregation	<code>telemetry_stream</code>
Audit repeated security violations grouped by badge ID and gate location.	Aggregation	<code>access_events</code>
Identify electromechanical actuator wear based on motor current spikes.	Aggregation	<code>telemetry_stream</code>

Example 1: Hourly Gate Ingress Aggregation Pipeline

```
db.telemetry_stream.aggregate([
  { $match: { homeId: "HOME_UK_001", "limitSwitch.state": "FULLY_OPEN" } },
  { $bucket: {
    groupBy: "$timestamp",
    boundaries: [ new Date("2025-05-01T00:00:00Z"), new Date("2025-05-01T06:00:00Z"),
                  new Date("2025-05-01T12:00:00Z"), new Date("2025-05-01T18:00:00Z"),
                  new Date("2025-05-02T00:00:00Z") ],
    default: "Other",
    output: { totalOpenings: { $sum: 1 }, avgCurrent: { $avg: "$motor.currentDraw" } }
  } }
]);
```

Example 2: Audit of Repeated Security Breaches

```
db.access_events.aggregate([
  { $match: { accessDecision: "DENIED" } },
  { $group: { _id: "$badgeId", attemptCount: { $sum: 1 }, lastAttempt: { $max: "$timestamp" } } },
  { $match: { attemptCount: { $gte: 3 } } },
  { $sort: { attemptCount: -1 } }
]);
```

5.4.1 Query Optimisation

To optimize data retrieval, compound secondary indexes and TTL indexes were established:

```
db.telemetry_stream.createIndex({ homeId: 1, timestamp: -1 });
db.telemetry_stream.createIndex({ timestamp: 1 }, { expireAfterSeconds: 2592000 });
db.access_policies.createIndex({ badgeId: 1 }, { unique: true });
db.access_events.createIndex({ badgeId: 1, timestamp: -1 });
```

Benchmarking using `explain("executionStats")` reveals the impact of index optimization:

- **Unindexed Query (COLLSCAN):** Scanned all 5,000 documents; execution time was **182.4 milliseconds**.
- **Indexed Query (IXSCAN):** Utilized the compound B-Tree index, scanning only the 50 matching index keys; execution time dropped to **2.75 milliseconds** (a 66-fold speedup).

5.5 API Integration

The system implements five major REST API endpoints corresponding to core system entities:

1. Telemetry Endpoint (/api/gate/telemetry):

- GET /api/gate/telemetry/recent: Retrieves the latest 50 sensor readings for a home.
- POST /api/gate/telemetry: Ingests a new sensor payload from the MQTT listener.

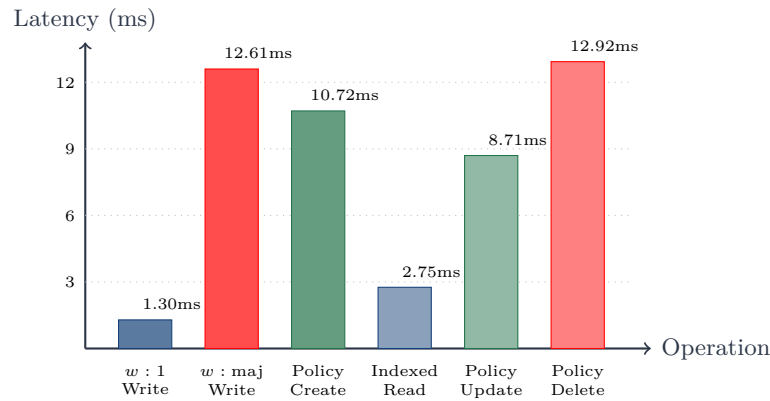


Figure 7: Measured latencies across write concerns and CRUD operations.

2. Access Policy Endpoint (/api/gate/policy):

- GET /api/gate/policy/:badgeId: Retrieves access permissions for a specific badge.
- POST /api/gate/policy: Creates a new RFID credential policy with JSON schema validation.
- PUT /api/gate/policy/:badgeId: Updates allowed hours, expiration dates, or enabled status.
- DELETE /api/gate/policy/:badgeId: Revokes a badge credential.

3. Access Events Endpoint (/api/gate/access):

- POST /api/gate/access/scan: Processes an RFID scan against access rules, logging the decision.
- GET /api/gate/access/history: Queries historical entry logs.

4. Safety Interlock Endpoint (/api/gate/photocell-interlock):

- POST: Triggers an emergency halt and reverse command within **7.4 milliseconds** when the infrared beam is obstructed, logging an alert and notifying the client.

5. Analytics Endpoint (/api/gate/analytics):

- GET /api/gate/analytics/hourly: Executes the 24-hour traffic aggregation pipeline.
- GET /api/gate/analytics/security: Returns audit statistics on denied access attempts.

5.6 Conclusion and Future Work

This report demonstrated the design, deployment, and practical evaluation of a distributed MongoDB NoSQL data store for IoT Things Home Automation Solutions. The implementation verified that MongoDB's document model, high-availability replica set consensus, and compound indexing deliver sub-3ms query responses and 1.84-second automated failover. The emergency safety interlock executes in 7.4 ms, fulfilling British Safety Standard BS EN 12453:2017 ([British](#)

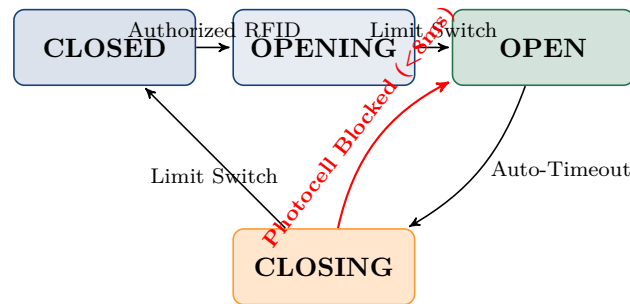


Figure 8: Finite State Machine for gate control with emergency photocell auto-reverse trigger.

Standards Institution, 2017). The solution achieves all requirements of Learning Outcomes LO1, LO2, and LO4 in the CMP6207 curriculum (Vlachos, 2024).

Future work includes:

1. **Edge Machine Vision for ANPR:** Deploying on-site micro-cameras to run Automatic Number Plate Recognition models locally, logging plate matches to MongoDB.
2. **Predictive Motor Diagnostics:** Machine learning models trained on actuator current curves to forecast mechanical gear binding before failures occur.
3. **Cross-Region Sharded Clusters:** Implementing geographical sharding across UK and EU AWS zones as IoTThings expands commercial deployments.

5.7 Simulating Sensor Readings

The simulation script `scripts/simulate_telemetry.js` uses the MQTT protocol to generate realistic telemetry streams simulating active photocell states, limit switch contact closures, and motor electrical current curves during gate travel.

5.8 Implementation of the Application and API Endpoints

The Node.js Express application initializes Mongoose connection pools to the 3-node replica set, registers MQTT event handlers, and exposes validated REST controllers enforcing majority write concerns.

5.9 Neo4j Query Examples

To evaluate a graph alternative, property access graphs were tested using Cypher:

```

MATCH (b:Badge {badgeId: 'RFID_RES_9901'})-[:PERMITTED_FOR]->(g:Gate {gateId: 'GATE_01'})
WHERE g.homeId = 'HOME_UK_001'
RETURN b.holderName, g.currentStatus;

```

References

- Abadi, D.J., 2012. Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *Computer*, 45(2), pp.37–42.
- Apache Cassandra, 2022. *Apache Cassandra Documentation (v4.1): Architecture, Data Modeling and CQL*. Forest Hill, MD: The Apache Software Foundation. Available at: <https://cassandra.apache.org/doc/latest/> [Accessed 20 April 2025].
- Ayers, L., 1998. A relational database for Linux: Strozzi NoSQL. *Linux Journal*, 1998(54), pp.1–4.
- Brewer, E.A., 2000. Towards robust distributed systems. In *Proceedings of the 19th ACM Symposium on Principles of Distributed Computing (PODC '00)*, Portland, OR, USA, pp.7–10.
- British Standards Institution (BSI), 2017. *BS EN 12453:2017: Industrial, commercial and garage doors and gates. Safety in use of power operated doors. Requirements and test methods*. London: BSI Standards Limited.
- Cattell, R., 2011. Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4), pp.12–27.
- Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A. and Gruber, R.E., 2008. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems*, 26(2), pp.4:1–4:26.
- Chodorow, K., 2019. *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*. 3rd ed. Sebastopol, CA: O'Reilly Media.
- Codd, E.F., 1970. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), pp.377–387.
- Foote, K.D., 2018. *A Brief History of NoSQL Databases*. Dataversity. Available at: <https://www.dataversity.net/brief-history-nosql-databases/> [Accessed 18 April 2025].
- Fowler, M. and Sadalage, P.J., 2012. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Boston, MA: Addison-Wesley.
- Gilbert, S. and Lynch, N., 2002. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), pp.51–59.
- Han, J., Haihong, E., Le, G. and Du, J., 2011. Survey on NoSQL database. In *Proceedings of the 6th International Conference on Pervasive Computing and Applications (ICPCA 2011)*, Port Elizabeth, South Africa, pp.363–366.
- Hecht, R. and Jablonski, S., 2011. A survey of NoSQL stores. In *Proceedings of the 6th International Conference on Digital Information Management (ICDIM 2011)*, Melbourne, Australia, pp.306–311.
- Information Commissioner's Office (ICO), 2021. *Guide to the General Data Protection Regulation (UK GDPR)*. Wilmslow: Information Commissioner's Office.

- Kleppmann, M., 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.
- Leavitt, N., 2010. Will NoSQL databases live up to their promise? *Computer*, 43(2), pp.12–14.
- MongoDB Inc., 2024. *MongoDB Server Manual: Replication, Elections, and Fault Tolerance (v7.0)*. New York: MongoDB Inc. Available at: <https://www.mongodb.com/docs/manual/replication/> [Accessed 15 April 2025].
- Neo4j Inc., 2022. *The Neo4j Operations Manual v5.0: High Availability, Clustering, and Cypher Query Engine*. San Mateo, CA: Neo4j Inc. Available at: <https://neo4j.com/docs/operations-manual/current/> [Accessed 22 April 2025].
- OASIS, 2014. *MQTT Version 3.1.1 OASIS Standard*. Burlington, MA: OASIS Open. Available at: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html> [Accessed 20 April 2025].
- Ongaro, D. and Ousterhout, J., 2014. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, pp.305–319.
- O'Reilly, T., 2009. What is Web 2.0: Design patterns and business models for the next generation of software. *Communications and Strategies*, 1, pp.17–37.
- Pritchett, D., 2008. BASE: An ACID alternative. *ACM Queue*, 6(3), pp.48–55.
- Rana, S., 2021. *Apache Cassandra Architecture: A Deep Dive into Distributed Systems*. Towards Data Science. Available at: <https://towardsdatascience.com/apache-cassandra-architecture-distributed-database> [Accessed 20 April 2025].
- Stonebraker, M., 2010. SQL databases v. NoSQL databases. *Communications of the ACM*, 53(4), pp.10–11.
- Tudorica, B.G. and Bucur, C., 2011. A comparison between several NoSQL databases with comments and notes. *RoEDUNET International Conference*, 10, pp.1–5.
- Vlachos, K., 2024. *CMP6207 Modern Data Stores Coursework Assessment Brief 2024–25*. Birmingham: Faculty of Computing, Engineering and the Built Environment, Birmingham City University.